МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна. Вариант 11.

Студент гр. 4345 Лунченко Н.С.
Преподаватель Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы:

Изучить и реализовать метод нахождения редакционного расстояния.

Постановка задачи

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.

Затем необходимо заменить два последних символа: stal -> stie.

Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L, равное расстоянию Левенштейна между строками S и T.

Вар. 11. Среди всех префиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти префиксы. Среди всех суффиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти суффиксы. Асимптотическая сложность решения задачи должна быть не больше, чем при обычном вычислении редакционного расстояния.

Выполнение работы

Таблица динамического программирования dp - матрица размером (N+1) × (M+1), где:

N - длина первой строки A

M - длина второй строки B

dp[i][j] - минимальная стоимость преобразования первых i символов строки A в первые j символов строки B

Такое представление позволяет вычислить редакционное расстояние за O(N·M) операций.

Заполняем матрицу динамического программирования по рекуррентной формуле:

```
dp[i][j] = min(
    dp[i-1][j] + 1,       // удаление
    dp[i][j-1] + 1,       // вставка
    dp[i-1][j-1] + replace_cost    // замена или совпадение
)
```

После вычисления таблицы dp для строк A и B, каждая ячейка dp[i][j] содержит минимальную стоимость превращения префикса A[:i] в префикс B[:j].

Нас интересует превращение префиксов A[:i] в полную строку B (длины M).

Эти значения находятся в последнем столбце таблицы: dp[i][M] для всех i от 0 до N.

Поиск минимальных суффиксов

Суффикс строки A - это подстрока, начинающаяся с некоторой позиции k и идущая до конца: A[k:]. Суффикс длины L строки A эквивалентен перевернутому префиксу длины L перевернутой строки A. Разберем для строки “hello”, перевернем её – “olleh”, возьмем префикс “olle”, перевернём и получим “ello” – суффикс первоначальной строки.

Асимптотическая сложность решения задачи не больше, чем при обычном вычислении редакционного расстояния, т.к. выполняется заполнения для двух таблиц – O(2·N·M) = O(N·M)

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

Входные данные	Выходные данные	Комментарии
asdfgh asd	Минимальное расстояние: 0 Префиксы с минимальным расстоянием: 'asd' Минимальное расстояние: 3 Суффиксы с минимальным расстоянием: (пустая строка) 'h' 'gh' 'fgh' 'asdfgh'	Строка B полностью входить в строку A. Минимальное расстояние у префикса asd – 0, т.к. он полностью совпадает со строкой B.
asd asdfg	Минимальное расстояние: 2 Префиксы с минимальным расстоянием: 'asd' Минимальное расстояние: 2	Минимальное расстояние у префикса asd – 2, Строка A полностью входит в начало строки B. Недостающие символы в B: 'f', 'g' (длина 2). С суффиксом asd та же ситуация.

	Суффиксы с минимальным расстоянием: 'asd'	
pedestal stien	Минимальное расстояние: 4 Префиксы с минимальным расстоянием: 'pe' 'ped' 'pede' 'pedes' Минимальное расстояние: 3 Суффиксы с минимальным расстоянием: 'stal'	pe -> 1 замена, 3 вставки ped -> 2 замены, 2 вставки pede -> 3 замены, одна ставка pedes -> 4 замены. stal -> 1 замена, 2 вставки
slukhi slukhi	Минимальное расстояние: 0 Префиксы с минимальным расстоянием: 'slukhi' Минимальное расстояние: 0 Суффиксы с минимальным расстоянием:	A = B

	'slukhi'	

Вывод.

Был изучен и реализован алгоритм Вагнера–Фишера для вычисления редакционного расстояния. Реализован поиск префиксов и суффиксов первой строки с минимальным расстоянием до второй строки. Для анализа суффиксов применён метод реверса строк, позволяющий свести задачу к анализу префиксов. Временная сложность алгоритма составляет $O(n \cdot m)$, что не превышает сложности стандартного вычисления расстояния Левенштейна.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл levenshtein.py

```
def levenshtein_distance(s1, s2):
    n, m = len(s1), len(s2)

    dp = [[0] * (m + 1) for i in range(n + 1)]

    for i in range(1, n + 1):
        dp[i][0] = i
        print(f"Для ячейки {i}, 0, лучший результат ячейка {i-1}, 0 +
удаление")
    for j in range(1, m + 1):
        dp[0][j] = j
        print(f"Для ячейки 0, {j}, лучший результат ячейка {0}, {j-1} +
вставка")
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = 0 if s1[i - 1] == s2[j - 1] else 1
            delete_cost = dp[i - 1][j] + 1
            insert_cost = dp[i][j - 1] + 1
            replace_cost = dp[i - 1][j - 1] + cost

            dp[i][j] = min(delete_cost, insert_cost, replace_cost)
            if delete_cost == dp[i][j]:
                print(f"Для ячейки {i}, {j}, лучший результат ячейка {i-
1}, {j} + удаление")
            elif insert_cost == dp[i][j]:
                print(f"Для ячейки {i}, {j}, лучший результат ячейка
{i}, {j-1} + вставка")
            elif cost == 0:
                print(f"{s1[i-1]} = {s2[j-1]}, ячейка {i}, {j} = {i-1},
{j-1} ")
            else:
                print(f"Для ячейки {i}, {j}, лучший результат ячейка {i-
1}, {j-1} + замена")

    for i in dp:
```

```
        print(i)
    return dp


def v_11():
    A = input().strip()
    B = input().strip()

    n, m = len(A), len(B)
    dp = levenshtein_distance(A, B)

    min_prefix_dist = float('inf')
    best_prefixes = []

    for i in range(n + 1):
        dist = dp[i][m]
        prefix = A[:i]

        if dist < min_prefix_dist:
            min_prefix_dist = dist
            best_prefixes = [prefix]
        elif dist == min_prefix_dist:
            best_prefixes.append(prefix)

    print(f"Минимальное расстояние: {min_prefix_dist}")
    print(f"Префиксы с минимальным расстоянием:")
    for p in best_prefixes:
        print(f"  '{p}'" if p else "  (пустая строка)")

    A_rev = A[::-1]
    B_rev = B[::-1]

    dp_rev = levenshtein_distance(A_rev, B_rev)

    min_suffix_dist = float('inf')
    best_suffixes = []

    for length in range(n + 1):
        dist = dp_rev[length][m]
```

```
        suffix = A[n - length:] if length > 0 else ""

        if dist < min_suffix_dist:
            min_suffix_dist = dist
            best_suffixes = [suffix]
        elif dist == min_suffix_dist:
            best_suffixes.append(suffix)

    print(f"Минимальное расстояние: {min_suffix_dist}")
    print(f"Суффиксы с минимальным расстоянием:")
    for s in best_suffixes:
        print(f"  '{s}'" if s else "  (пустая строка)")


if __name__ == "__main__":
    v_11()
```